Methodology for Stress-Testing Recommender Systems with Edge-Case User Profiles

Mattes Jendrik Wichert
Johannes Kepler University Linz
Linz, Austria
m.wichert2003@gmail.com

Jeevan Murali
Johannes Kepler University Linz
Linz, Austria

Agampreet Singh
Johannes Kepler University Linz
Linz, Austria

Ashutush Choudhury
Johannes Kepler University Linz
Linz, Austria

Norwin Wilden
Johannes Kepler University Linz
Linz, Austria

Abstract

This paper defines the methodology for investigating how recommender system model families behave under test-time edge-case user profiles. Building on our previous literature review, we focus on two practically feasible edge-case scenarios: niche users identified within a dataset and perturbation-based user profiles generated from real user histories. The planned study compares several recommender model families on MovieLens using both standard ranking metrics and robustness-oriented metrics such as subgroup performance gaps, recommendation stability, and popularity bias. The goal of this methodology paper is not to report final experimental results, but to specify a rigorous and reproducible experimental design.

Keywords

recommender systems, robustness, edge-case users, niche users, perturbation, evaluation

1 Introduction

Recommender systems are commonly evaluated using average ranking performance across all users. While such evaluation is useful, it can hide weaknesses for atypical or underrepresented user groups. In this project, we investigate whether recommender models behave differently when evaluated on edge-case user profiles at test time.

The focus of this methodology paper is to define an experimental setup for stress-testing recommender systems on two types of edge-case users. First, we consider niche users, i.e., users whose interaction histories contain a relatively high proportion of less popular or long-tail items. Second, we consider perturbation-based user profiles, where real user histories are modified in a small and controlled way to test whether model recommendations remain stable or shift in response to realistic profile changes.

2 Updated Research Questions

The first research question remains the central motivation of the project:

RQ1: How do different recommender model families behave when evaluated on edge-case user profiles at test time?

Based on feedback on the initial research questions, the second and third research questions will be refined to be more concrete and manageable. Instead of proposing a broad universal benchmark, the revised questions will focus on reproducible edge-case identification and perturbation-based evaluation.

Potential revised research questions are:

RQ2: How can niche users be identified in a recommender dataset in a reproducible way that is suitable for comparing different model families?

RQ3: How stable are recommender model outputs when real user profiles are modified through small, realistic, interaction-based perturbations?

These revised questions keep the project feasible while still addressing the broader goal of edge-case recommender evaluation.

3 Proposed Experimental Setup

The planned experimental setup is intentionally kept manageable. We plan to start with one main dataset, MovieLens, and compare three types of recommender systems. The evaluation will focus on two edge-case user scenarios: niche users found in the dataset and perturbation-based profiles generated from real users.

3.1 Dataset

The main dataset will be MovieLens. MovieLens is suitable for this project because it contains user-item interactions, ratings, timestamps, and item metadata. It is also widely used in recommender systems research, making it a practical choice for a methodology-focused study.

In the initial setup, MovieLens will be used as the core dataset. A second dataset may be considered as an optional extension if time allows. The purpose of a second dataset would be to test whether the proposed evaluation logic can transfer across datasets with different sparsity, popularity distributions, or item domains.

3.2 Models

The planned model comparison will include different recommender model families. The exact implementations still need to be finalized, but the candidate models are:

- a popularity-based baseline,
- a classical collaborative filtering model such as matrix factorization,
- a neural collaborative filtering model,
- optionally, a sequential recommender such as SASRec.

The models should ideally be trained by the project team using the same train, validation, and test splits. This avoids data leakage

and ensures that edge-case users are evaluated consistently across model families. If pretrained models are used, the training data and split procedure must be known and verified to prevent leakage.

4 Preprocessing and Splitting Strategy

The dataset will be preprocessed into user-item interactions. Depending on the final decision, ratings may either be treated as explicit feedback or converted into implicit positive interactions. The preprocessing should include user and item filtering to remove users or items with too few interactions.

A consistent train, validation, and test split will be used for all models. A temporal split is preferred because recommender systems usually predict future interactions from past behavior. Under this setup, each user's earlier interactions are used for training and later interactions are used for validation and testing.

Special care must be taken to avoid data leakage. Item popularity, niche-user identification, similarity computation, and perturbation candidate selection should be based only on the training data or on the observable user history available at test time, not on future test interactions.

5 Edge-Case User Construction

This project considers two test-time edge-case user groups: niche users and perturbation-based users.

5.1 Niche Users

Niche users will be identified based on their interaction history. A possible strategy is to compute item popularity from the training set, define long-tail items using a relative popularity threshold, and then compute each user's long-tail interaction ratio. Users with high long-tail ratios can be treated as niche users.

It is important to distinguish niche users from sparse users. A user should not be considered niche merely because they have very few interactions. Therefore, the niche-user definition should include a minimum history length and should compare niche users against a suitable mainstream or non-niche control group.

5.2 Perturbation-Based Users

Perturbation-based users will be generated from real user profiles rather than created manually from scratch. The goal is to apply small, controlled, and realistic modifications to user histories at test time. These modified profiles can then be used to measure how sensitive or stable different recommender models are.

A possible perturbation strategy is interaction-based replacement. For an eligible user, we select one item from the observed history and replace it with another item that is less popular but still similar according to the user-item interaction matrix. Similarity can be computed using interaction-based measures such as co-occurrence, Jaccard similarity, cosine similarity, lift, or pointwise mutual information. This avoids relying on dataset-specific metadata such as movie genres and makes the perturbation logic more transferable across datasets.

The perturbation should satisfy several constraints. The replacement item should not already appear in the user's history, should not be part of the user's future test target items, should be less popular than the replaced item, and should be sufficiently similar to the original item or the user's profile. Instead of using a fixed absolute similarity threshold, candidate items may be selected based on similarity ranks or percentiles, such as top-5%, top-20%, or top-40% most similar candidates.

This design allows different perturbation strengths to be tested. Highly similar replacements can serve as a stability control, while less popular but still related replacements can test whether the model reacts to a weak niche signal. The perturbation should remain small, for example by changing only one item or at most 5–10% of the user history.

6 Evaluation Protocol and Metrics

The evaluation will be conducted at several levels. First, standard ranking performance will be measured on all users to ensure that each model works under normal conditions. Second, the same metrics will be computed separately for edge-case users, including niche users and perturbation-based profiles.

6.1 Standard Ranking Metrics

The planned standard ranking metrics include:

- Recall@K,
- NDCG@K,
- Hit Rate@K,
- Precision@K.

The final values of K still need to be selected, for example K=10 or K=20.

6.2 Subgroup Performance

For edge-case evaluation, standard ranking metrics will be computed separately for all users, mainstream users, niche users, and perturbed users. A central quantity is the performance gap between mainstream and niche users:

$$Gap@K = Metric@K_{mainstream} - Metric@K_{niche}. \tag{1}$$

This helps reveal whether average performance hides weaker performance for niche users.

6.3 Robustness and Stability Metrics

For perturbation-based profiles, the evaluation will compare recommendations before and after perturbation. Possible metrics include performance drop and recommendation list overlap:

$$\Delta NDCG@K = NDCG@K_{original} - NDCG@K_{perturbed}. \tag{2}$$

$$Overlap@K = \frac{|TopK_{original} \cap TopK_{perturbed}|}{K}. \tag{3}$$

These metrics measure whether small profile changes lead to large changes in recommendation quality or recommendation content.

6.4 Popularity Bias and Niche Alignment

Because niche users are central to the project, the evaluation should also measure whether models fall back to popular items. Planned

metrics include average recommendation popularity, long-tail recommendation ratio, and popularity mismatch between the user's history and the generated recommendation list.

7 Ablation Study

Several ablations may be used to test whether the results depend on specific design choices. Possible ablations include:

- varying the niche-user threshold,
- comparing different perturbation strengths,
- replacing one item versus multiple items,
- comparing similarity measures such as cosine similarity, Jaccard similarity, lift, and PMI,
- evaluating whether perturbing early, middle, or recent interactions changes model behavior.

8 Potential Challenges and Mitigation Strategies

Several challenges are expected. First, niche users may be difficult to define without confusing them with sparse users. This can be mitigated by requiring a minimum user-history length and comparing niche users with matched mainstream users. Second, interaction-based similarity may be unstable for extremely rare items. This can be mitigated by excluding items with too few interactions from the perturbation candidate pool. Third, training all models may be computationally expensive. This can be handled by starting with MovieLens and using small, reproducible model configurations.

Another challenge is data leakage. Popularity statistics, similarity scores, and perturbation candidates must be computed without using future test interactions. Finally, perturbation-based profiles must remain realistic. This can be addressed through constraints on perturbation size, similarity rank, and replacement-item popularity.

9 Overall Experimental Pipeline

The planned workflow is:

(1) Select and preprocess MovieLens.
(2) Create train, validation, and test splits.
(3) Train recommender models on the training data.
(4) Compute item popularity and item-item similarity from training interactions.
(5) Identify niche users in the test set.
(6) Generate perturbation-based user profiles from eligible real users.
(7) Evaluate all models on normal users, niche users, and perturbed users.
(8) Compare standard performance, subgroup gaps, stability, robustness, and popularity bias.

10 Expected Outcomes

We expect that average-case performance may not fully reflect model behavior on edge-case users. Some models may perform well on all users on average but show larger performance gaps for niche users. We also expect different model families to react differently to perturbations. For example, sequential models may be more sensitive to recent perturbations, while non-sequential collaborative filtering models may be more stable but less adaptive.

11 Disclosure of LLM Assistance

Large Language Models, including ChatGPT, were used during the preparation of this assignment for brainstorming, structuring the methodology, refining research questions, and drafting initial text passages. All LLM-assisted content will be reviewed and edited by the authors. Methodological claims, references, formulas, and final design decisions will be checked against the relevant literature and implementation constraints before submission.